



Hands-On With the Neo4j Graph Platform Session 8

Class Agenda

- Labeled Property Graph (LPG) Components 🙌
- What is a graph database? 🙌
- What is Neo4j? 🙌
- Common Graph Use Cases 🙌
- Graph modeling -Part 1 🙌

- Graph modeling -Part 2 🙌
- Importing CSV into Neo4j

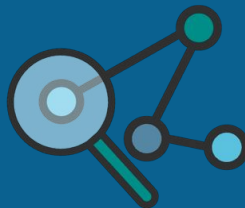


Constraints and Indexes

- Intro to Graph Data Science
- Graph Data Science workflows



Hands-On





Constraints



Unique IDs and Constraints



Syntax

```
CREATE CONSTRAINT [name] [IF NOT EXISTS]
FOR (x:LabelName)
  REQUIRE x.propertyName IS [NODE] UNIQUE
[OPTIONS "{" option: value[, ...] "}"]
```

Benefits

- **Duplicate** data is **not created**
- **Prevent problems** creating relationships between nodes
- Creating a constraint **automatically** also creates an **index**

Types of Constraints

5 (as of v5.22) Types of Constraints (implemented as an index)

- **Uniqueness**
 - Property must be unique
- **Existence**
 - Property must have a value
- **Data type constraints (5.9/5.11)**
 - Property must be of a datatype
- **Node Key**
 - Must exist and be unique
- **Relationship Key (5.7)**
 - Must exist and be unique

Creating Uniqueness Constraints

Single property syntax

```
CREATE CONSTRAINT <constraint_name>  
[IF NOT EXISTS]  
FOR (x:<node_label>)  
  REQUIRE x.<property_key> IS UNIQUE
```

Creating a constraint **after** data is loaded may **fail** if the data does not meet the constraint

Multiple properties syntax

```
CREATE CONSTRAINT <constraint_name>  
[IF NOT EXISTS]  
FOR (x:<node_label>)  
  REQUIRE (x.<property_key1>, x.<property_key2>) IS UNIQUE
```

Creating Existence Constraints

Single property syntax

```
CREATE CONSTRAINT <constraint_name>  
[IF NOT EXISTS]  
FOR (x:<node_label>)  
  REQUIRE x.<property_key> IS NOT NULL
```

Creating a constraint **after** data is loaded may **fail** if the data does not meet the constraint

Multiple properties syntax

```
CREATE CONSTRAINT <constraint_name>  
[IF NOT EXISTS]  
FOR (x:<node_label>)  
  REQUIRE (x.<property_key1>, x.<property_key2>) IS NOT NULL
```

Creating Node Property Type Constraints (5.9/5.11)

Syntax

```
CREATE CONSTRAINT <constraint_name>  
[IF NOT EXISTS]  
FOR (x:<node_label>)  
  REQUIRE x.<property_key> IS :: <TYPE>
```

Creating a constraint **after** data is loaded may **fail** if the data does not meet the constraint

Example

```
CREATE CONSTRAINT person_name  
[IF NOT EXISTS]  
FOR (x:Person)  
  REQUIRE x.name IS :: STRING
```

<https://neo4j.com/docs/cypher-manual/current/constraints/syntax/#constraints-syntax-create-node-prop-type>

Creating Node Key Constraints

Single property syntax

```
CREATE CONSTRAINT <constraint_name>
[IF NOT EXISTS]
FOR (x:<node_label>)
  REQUIRE x.<propertyName> IS NODE KEY
```

Creating a constraint **after** data is loaded may **fail** if the data does not meet the constraint

Multiple properties syntax

```
CREATE CONSTRAINT <constraint_name>
[IF NOT EXISTS]
FOR (x:<node_label>)
  REQUIRE (x.<propName1>,x.<propName2>) IS NODE KEY
```

Creating Relationship Key Constraints (5.7)

Single property syntax

```
CREATE CONSTRAINT <constraint_name>  
[IF NOT EXISTS]  
FOR ()-[x:REL_TYPE]-()  
REQUIRE x.<propertyName> IS [REL[ATIONSHIP]] KEY
```

Creating a constraint **after** data is loaded may **fail** if the data does not meet the constraint

Multiple properties syntax

```
CREATE CONSTRAINT <constraint_name>  
[IF NOT EXISTS]  
FOR ()-[x:REL_TYPE]-()  
REQUIRE (x.<propName1>,x.<propName2>) IS [REL[ATIONSHIP]] KEY
```

Managing Constraints

Creating constraints

```
CREATE CONSTRAINT...
```

Listing constraints

```
SHOW CONSTRAINTS
```

Drop constraints

```
DROP CONSTRAINT <constraint_name>
```

Dropping Constraints

Drop a single constraint

```
DROP CONSTRAINT <constraint_name> [IF EXISTS]
```

Creating drop statements

```
SHOW CONSTRAINTS YIELD name  
RETURN collect('DROP CONSTRAINT ' + name + ';' ) as statements
```

Dropping all constraints

```
CALL apoc.schema.assert({}, {}, true)
```



Indexes



Best Practice for Loading Data



- 1 Create constraints
- 2 Load data
- 3 Create indexes

Indexes makes
writing data **slower** but
reading data **faster**

- Indexes are a data structure for quick retrieval of data
- Requires more storage

Do not index
everything

Search Performance Indexes

Indexes used for exact matches

1

Lookup Indexes

Built-in index on nodes and relationship types

2

Range & Composite Indexes

General purpose index for single and multiple properties

3

Text Indexes

Optimize queries on string properties

4

Point Indexes

Improve predicates operating on geospatial values

<https://neo4j.com/docs/cypher-manual/current/indexes/search-performance-indexes/using-indexes/>

Lookup Indexes

1 Created automatically on nodes (labels) and relationships (relationship types)

2 **NEVER** delete them

```
neo4j$ show indexes
```

	id	name	state	populationPercent	type	entityType	labelsOrTypes	properties	indexProvider
1	0	"index_343aff4e"	"ONLINE"	100.0	"LOOKUP"	"NODE"	<i>null</i>	<i>null</i>	"token-lookup-1.0"
2	1	"index_f7700477"	"ONLINE"	100.0	"LOOKUP"	"RELATIONSHIP"	<i>null</i>	<i>null</i>	"token-lookup-1.0"

RANGE Index

- Proprietary **BTree** implementation
- Any datatype
- Equality checks: =
 - a Text index might be better for string properties
- Range comparisons: >, >=, <, <=
- String comparisons: STARTS WITH
- Existence checks: IS NOT NULL

Syntax

```
CREATE INDEX <index-name>  
IF NOT EXISTS  
FOR (x:LabelName)  
ON (x.propertyName)
```

```
// Index on Rel property  
CREATE INDEX <index-name>  
IF NOT EXISTS  
FOR ()-[x:Relationship]-()  
ON (x.propertyName)
```

Index not created if an index for the same Node or Relationship and property

Composite Index

- RANGE index on multiple properties of a node or relationship
- Always tested together in a query
- Properties can be of mixed data types

Syntax

```
CREATE INDEX <index-name>  
IF NOT EXISTS  
FOR (x:LabelName)  
ON (x.property1, x.property2)
```

```
MATCH (m:Movie) WHERE m.year > 1999  
AND m.title CONTAINS 'Toy'  
RETURN m.title, m.year, m.imdbRating
```

A composite index on title and year would improve this query

TEXT Index (5.x)

- Property type must be a string
- Improves
 - Equality checks: =
 - String comparisons: ENDS WITH, CONTAINS
 - List membership: x.prop in ["a", "b", "c"]
- TEXT vs RANGE
 - Perform better than RANGE if there are many duplicate properties
 - Take up less space in the graph
- **NOT** used for:
 - Existence checks: IS NOT NULL

Syntax

```
CREATE TEXT INDEX <index-name>  
IF NOT EXISTS  
FOR (x:LabelName)  
ON (x.property1, x.property2)
```

```
MATCH (m:Person)  
WHERE p.name CONTAINS 'Jim'  
RETURN p
```

A TEXT index on name
would improve this
query

Profiling Queries

1

```
PROFILE MATCH (p:Person)
2 WHERE p.name STARTS WITH "Austin"
3 RETURN p.name
```

Query planner performs a LabelScan across all of the data

▼ NodeByLabelScan@neo4j	
p	
p:Person	
248 memory (bytes)	
55,509 pagecache hits	
0 pagecache misses	
19,047 estimated rows	
19,048 db hits	

19,047 rows

▼ Filter@neo4j	
p	
cache[p.name] STARTS WITH \$autostring_0	
571 estimated rows	
38,094 db hits	

3 rows

▼ Projection@neo4j	
p, `p.name`	
cache[p.name] AS `p.name`	
571 estimated rows	

Cypher version: 5, planner: COST, runtime: PIPELINED. 57142 total db hits in 58 ms.

2

Create the appropriate index

3

```
PROFILE MATCH (p:Person)
2 WHERE p.name STARTS WITH "Austin"
3 RETURN p.name
```

Confirm query is using the index

▼ NodeIndexSeekByRange@neo4j	
p	
RANGE INDEX p:Person(name) WHERE name STARTS WITH \$autostring_0, cache[p.name]	
248 memory (bytes)	
0 pagecache hits	
3 pagecache misses	
57 estimated rows	
14 db hits	

3 rows

▼ Projection@neo4j	
p, `p.name`	
cache[p.name] AS `p.name`	
57 estimated rows	
0 db hits	

3 rows

▼ ProduceResults@neo4j	
p, `p.name`	
`p.name`	
312 total memory (bytes)	
0 memory (bytes)	
57 estimated rows	

Cypher version: 5, planner: COST, runtime: PIPELINED. 14 total db hits in 30 ms.

Semantic Indexes

1

Fulltext Indexes

Supports Lucene queries

2

Vector Indexes

Returns a score indicating the similarity between the search string and the data in the DB

Full-Text Index Use Cases

- Complex patterns and string patterns
- Expensive using RANGE or TEXT indexes
- Complex Examples
 - All movie titles that contain “Night” followed by “Sky” later in the title
 - All movie plots that have “murder” but not “drugs”
 - All movie titles that contain “Night” and “Dead” that have a plot that contains “French”

Query planner uses 1 index (subqueries can use another)

Full-Text Index

- Useful for queries that must parse a string property value
- Uses Apache Lucene
- Use Lucene's query language to retrieve data
- Explicit call to use the index

Can be used to:

- Index multiple properties for a label or relationship type
- Index multiple labels or relationship types

Syntax

```
CREATE FULLTEXT INDEX <index-name>  
IF NOT EXISTS  
FOR (x:LabelName)  
ON EACH [x.property1]
```

```
// Multiple labels & properties  
CREATE FULLTEXT INDEX <index-name>  
IF NOT EXISTS  
FOR (x:Label1 | Label2)  
ON EACH [x.prop1, x.prop2,...]
```

Creating and Using Full-Text Indexes

- CREATE the full-text index
- CALL `db.index.fulltext.queryNodes()` to retrieve nodes
- CALL `db.index.fulltext.queryRelationships()` to retrieve relationships

```
CALL db.index.fulltext.queryNodes ('Movie_plot_ft', 'murder AND drugs')  
YIELD node, score  
RETURN node.title, node.plot
```


Vector Indexes (5.13)

Used in GenAI/LLM use cases for similarity searches

1. Split into chunks. Create embeddings (LLM, Python, etc).
 - Unstructured text (PDF, etc)
 - Paragraphs of text
 - User's question via chatBot implementation
2. Store embedding on nodes or properties
 - **db.create.setNodeVectorProperty()**
 - **db.create.setRelationshipVectorProperty()**
 - Stored as a LIST<INTEGER | FLOAT>
3. Create the vector index
 - See syntax
4. Use index for semantic search into Neo4j
 - Anchor query on text similar to another embedding

Syntax

```
CREATE VECTOR INDEX <index-name>
IF NOT EXISTS
FOR (x:LabelName)
ON (x.property1)
OPTIONS { indexConfig: {
  'vector.dimensions': 1536,
  'vector.similarity_function': 'cosine'
}}
```

Cosine and
euclidean

Query Vector Indexes (5.13)

Find 5 movies with the most similar plot as the Godfather

```
MATCH (m:Movie {title: 'Godfather, The'})  
CALL db.index.vector.queryNodes ('moviePlots', 5, m.embedding)  
YIELD node as movie, score  
RETURN movie.title, movie.plot
```

Use the
embedding from
movie node

Top 10 Products similar to a search term

```
CALL db.index.vector.queryNodes ('productDesc', 10, $queryVector)  
YIELD node as product, score  
RETURN product.productCode, product.productTypeName,  
       product.text, score ORDER BY score DESC
```

Embedding of
search phrase

Managing Indexes

Creating indexes

```
CREATE [RANGE|TEXT|FULLTEXT|POINT|LOOKUP] INDEX...
```

Listing indexes

```
SHOW [RANGE|TEXT|FULLTEXT|POINT|LOOKUP] INDEXES [YIELD *]
```

Drop indexes

```
DROP INDEX <index_name>
```

Additional Considerations

- Creating index (IF NOT EXISTS)
 - Will not create if index the index name already exists (duplicate)
 - Will not create if an index already exists with the same node/relationship properties
- Using toLower() or toUpper(), the query planner will NOT use the TEXT index
MATCH (n:Movie)
WHERE n.plot CONTAINS "smith" or n.plot CONTAINS "SMITH"
- Query planner only uses a single index for a query
 - Subqueries (CALL { }) can use an additional index
 - Can specify [query hint \(USING INDEX\)](#) to control the index being used
 - Results may be slower; Test query hints to ensure performance